Introduction :-

TensorFlow is an open-source machine learning and deep learning library developed by **Google**. It provides a comprehensive platform for building, training, and deploying machine learning models.

TensorFlow is widely used to create applications involving **Artificial Intelligence (AI)**, **Deep Learning (DL)**, **Computer Vision**, **Natural Language Processing (NLP)**, and many other intelligent systems.

It supports execution on **CPUs, GPUs, and TPUs**, making it suitable for both small projects and large-scale production systems.

Key Points :-

- Developed by Google.
- Open-source machine learning framework.
- Supports deep learning and neural networks.
- Compatible with multiple platforms and devices.
- One of the most popular AI frameworks in the industry.

Double-click (or enter) to edit

History :-

TensorFlow was developed by **Google Brain**, a team of researchers at Google focused on artificial intelligence and machine learning. It was officially released as an **open-source library in November 2015**.

TensorFlow was created as the successor to Google's earlier machine learning system called **DistBelief**. It was designed to be faster, more flexible, and capable of running efficiently on CPUs, GPUs, and TPUs.

Over the years, TensorFlow has become one of the most widely used deep learning frameworks in the world. It is used by researchers, students, developers, and organizations to build AI-powered applications.

Timeline -

- **2011:** Google developed DistBelief, an internal deep learning framework.
- **2015:** TensorFlow was released as an open-source project.
- **2017:** TensorFlow introduced Keras integration for easier model building.
- **2019:** TensorFlow 2.0 was released with simpler APIs and eager execution enabled by default.
- **Present:** TensorFlow is widely used for machine learning, deep learning, and AI applications across various industries.

Why TensorFlow was Developed:-

TensorFlow was developed to provide a powerful, flexible, and scalable platform for building machine learning and deep learning models. Before TensorFlow, creating and deploying large-scale AI models was more difficult and less efficient.

Google designed TensorFlow to simplify the process of developing intelligent applications while supporting high-performance computation across different hardware platforms.

Objectives of TensorFlow

- Simplify the development of machine learning and deep learning models.
- Support efficient computation on CPUs, GPUs, and TPUs.
- Enable easy deployment of trained models across different platforms.
- Provide a scalable framework for handling large datasets.
- Offer an open-source platform that encourages research and innovation.

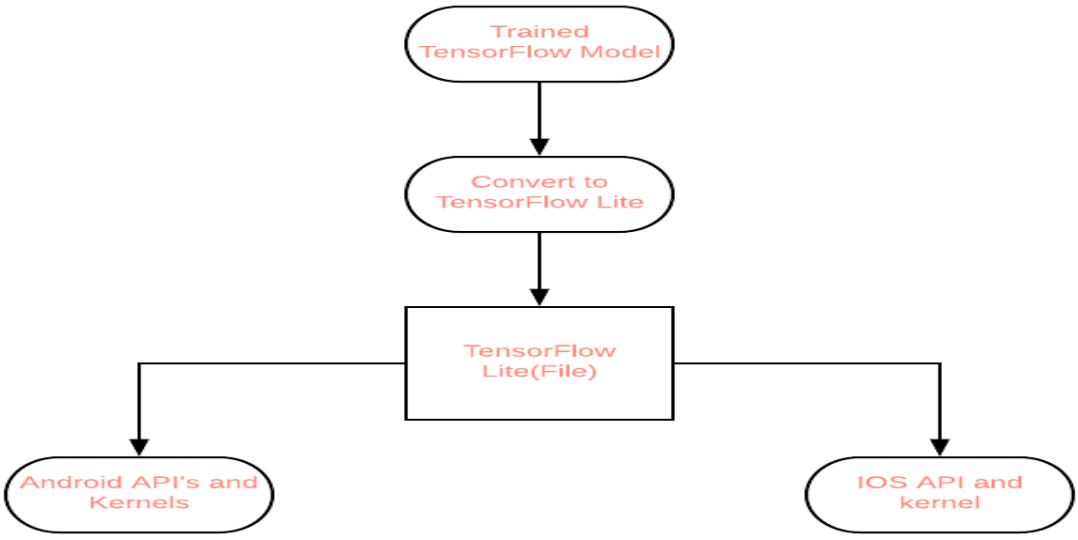
Features :-

- **Open Source:** Freely available and supported by a large developer community.
- **High Performance:** Optimized to run efficiently on CPUs, GPUs, and TPUs.
- **Scalable:** Suitable for both small projects and large-scale production systems.

- **Easy Model Building:** Integrates with Keras for creating neural networks with minimal code.
- **Automatic Differentiation:** Automatically computes gradients for training neural networks.
- **Cross-Platform Support:** Models can be deployed on web, mobile, cloud, and embedded devices.
- **Built-in Tools:** Provides tools for data processing, model training, evaluation, and deployment.

Applications:-

- **Image Classification:** Identifying objects, animals, or people in images.
- **Object Detection:** Detecting and locating multiple objects within an image or video.
- **Natural Language Processing (NLP):** Building chatbots, language translation systems, and text analysis applications.
- **Speech Recognition:** Converting spoken language into text for voice assistants.
- **Recommendation Systems:** Suggesting movies, products, music, or videos based on user preferences.
- **Medical Diagnosis:** Assisting doctors in detecting diseases from medical images and patient data.



Installation & Import:-

Before using TensorFlow, you need to install it on your system. After installation, you can import the library into your Python program or Google Colab notebook.

TensorFlow can be installed using the **pip** package manager. In Google Colab, TensorFlow is usually pre-installed.

1. Install TensorFlow:-

Use the following command to install the latest version of TensorFlow:

Command -

pip install tensorflow

2. Import TensorFlow:-

After installation, import TensorFlow using the following statement:

Command -

import tensorflow as tf

3. Check Installed Version:-

You can verify the installed version of TensorFlow using:

Command -

print(tf.**version**)

```
!pip install tensorflow
```

```
Requirement already satisfied: tensorflow in /usr/local/lib/python3.12/dist-p
Requirement already satisfied: absl-py>=1.0.0 in /usr/local/lib/python3.12/di
Requirement already satisfied: astunparse>=1.6.0 in /usr/local/lib/python3.12
Requirement already satisfied: flatbuffers>=24.3.25 in /usr/local/lib/python3
Requirement already satisfied: gast!=0.5.0,!=0.5.1,!=0.5.2,>=0.2.1 in /usr/lo
Requirement already satisfied: google_pasta>=0.1.1 in /usr/local/lib/python3.
Requirement already satisfied: libclang>=13.0.0 in /usr/local/lib/python3.12/
Requirement already satisfied: opt_einsum>=2.3.2 in /usr/local/lib/python3.12
Requirement already satisfied: packaging in /usr/local/lib/python3.12/dist-pa
Requirement already satisfied: protobuf>=5.28.0 in /usr/local/lib/python3.12/
Requirement already satisfied: requests<3,>=2.21.0 in /usr/local/lib/python3.
Requirement already satisfied: setuptools in /usr/local/lib/python3.12/dist-p
Requirement already satisfied: six>=1.12.0 in /usr/local/lib/python3.12/dist-
Requirement already satisfied: termcolor>=1.1.0 in /usr/local/lib/python3.12/
Requirement already satisfied: typing_extensions>=3.6.6 in /usr/local/lib/pyt
Requirement already satisfied: wrapt>=1.11.0 in /usr/local/lib/python3.12/dis
Requirement already satisfied: grpcio<2.0,>=1.24.3 in /usr/local/lib/python3.
Requirement already satisfied: tensorboard~=2.20.0 in /usr/local/lib/python3.
Requirement already satisfied: keras>=3.10.0 in /usr/local/lib/python3.12/dis
Requirement already satisfied: numpy>=1.26.0 in /usr/local/lib/python3.12/dis
Requirement already satisfied: h5py>=3.11.0 in /usr/local/lib/python3.12/dist
Requirement already satisfied: ml_dtypes<1.0.0,>=0.5.1 in /usr/local/lib/pyth
Requirement already satisfied: wheel<1.0,>=0.23.0 in /usr/local/lib/python3.1
```

```
Requirement already satisfied: rich in /usr/local/lib/python3.12/dist-package
Requirement already satisfied: namex in /usr/local/lib/python3.12/dist-packag
Requirement already satisfied: optree in /usr/local/lib/python3.12/dist-packa
Requirement already satisfied: charset_normalizer<4,>=2 in /usr/local/lib/pyt
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.12/dist
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.1
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.1
Requirement already satisfied: markdown>=2.6.8 in /usr/local/lib/python3.12/d
Requirement already satisfied: pillow in /usr/local/lib/python3.12/dist-packa
Requirement already satisfied: tensorboard-data-server<0.8.0,>=0.7.0 in /usr/
Requirement already satisfied: werkzeug>=1.0.1 in /usr/local/lib/python3.12/d
Requirement already satisfied: markupsafe>=2.1.1 in /usr/local/lib/python3.12
Requirement already satisfied: markdown-it-py>=2.2.0 in /usr/local/lib/python
Requirement already satisfied: pygments<3.0.0,>=2.13.0 in /usr/local/lib/pyth
Requirement already satisfied: mdurl~=0.1 in /usr/local/lib/python3.12/dist-p
```

```
import tensorflow as tf
```

```
print(tf.__version__)
```

```
2.20.0
```

Tensor Basics:-

A **Tensor** is the fundamental data structure in TensorFlow. It is similar to a NumPy array but is optimized for high-performance computations and can run efficiently on CPUs, GPUs, and TPUs.

Tensors can store numbers, vectors, matrices, and higher-dimensional data.

1. tf.constant():-

`tf.constant()` is used to create an immutable (unchangeable) tensor. Once created, its value cannot be modified.

Example:

tensor = tf.constant([10, 20, 30])

```
import tensorflow as tf

tensor = tf.constant([10, 20, 30])

print(tensor)
```

```
tf.Tensor([10 20 30], shape=(3,), dtype=int32)
```

2. tf.Variable():-

`tf.Variable()` is used to create a mutable (changeable) tensor. Variables are mainly used while training machine learning models because their values can be updated.

Example:

variable = tf.Variable([1, 2, 3])

```
import tensorflow as tf

variable = tf.Variable([1, 2, 3])

print(variable)
```

```
<tf.Variable 'Variable:0' shape=(3,) dtype=int32, numpy=array([1, 2, 3], dtyp
```

3. Tensor Properties:-

TensorFlow provides several properties to get information about a tensor.

- **Shape:** Returns the dimensions of the tensor.
- **Data Type (`dtype`):** Returns the type of data stored.
- **Number of Dimensions (`ndim`):** Returns the rank (number of dimensions).

```
import tensorflow as tf

tensor = tf.constant([[1, 2], [3, 4]])

print("Shape:", tensor.shape)
print("Data Type:", tensor.dtype)
print("Dimensions:", tensor.ndim)
```

```
Shape: (2, 2)
Data Type: <dtype: 'int32'>
Dimensions: 2
```

Tensor Operations :-

TensorFlow provides various operations to perform mathematical and matrix computations on tensors. These operations are optimized for high-performance execution on CPUs, GPUs, and TPUs.

Tensor operations are similar to NumPy operations but are specifically designed for machine learning and deep learning tasks.

1. tf.add():-

tf.add() is used to add two tensors element-wise.

Example:

tf.add(a, b)

```
import tensorflow as tf

a = tf.constant([10, 20, 30])
b = tf.constant([1, 2, 3])

result = tf.add(a, b)

print(result)
```

```
tf.Tensor([11 22 33], shape=(3,), dtype=int32)
```

2. tf.subtract():-

tf.subtract() is used to subtract one tensor from another element-wise.

Example:

tf.subtract(a, b)

```
import tensorflow as tf

a = tf.constant([10, 20, 30])
b = tf.constant([1, 2, 3])

result = tf.subtract(a, b)

print(result)
```

```
tf.Tensor([ 9 18 27], shape=(3,), dtype=int32)
```

3. tf.multiply():-

tf.multiply() is used to multiply two tensors element-wise.

Example:

tf.multiply(a, b)

```
import tensorflow as tf

a = tf.constant([10, 20, 30])
b = tf.constant([1, 2, 3])
```

```
result = tf.multiply(a, b)

print(result)
```

```
tf.Tensor([10 40 90], shape=(3,), dtype=int32)
```

4. tf.divide():-

tf.divide() is used to divide one tensor by another element-wise.

Example:

tf.divide(a, b)

```
import tensorflow as tf

a = tf.constant([10.0, 20.0, 30.0])
b = tf.constant([2.0, 4.0, 5.0])

result = tf.divide(a, b)

print(result)
```

```
tf.Tensor([5. 5. 6.], shape=(3,), dtype=float32)
```

5. tf.matmul():-

tf.matmul() performs matrix multiplication between two tensors.

Example:

tf.matmul(a, b)

```
import tensorflow as tf

a = tf.constant([[1, 2],
                 [3, 4]])

b = tf.constant([[5, 6],
                 [7, 8]])

result = tf.matmul(a, b)

print(result)
```

```
tf.Tensor(
[[19 22]
 [43 50]], shape=(2, 2), dtype=int32)
```

Neural Network Basics :-

A **Neural Network** is a machine learning model inspired by the structure and working of the human brain. It consists of interconnected nodes called **neurons**, which process data and learn patterns from it.

Neural networks are mainly used for solving complex problems such as image recognition, speech recognition, natural language processing, and prediction tasks.

Components of a Neural Network :-

1. Input Layer:-

The Input Layer is the first layer of a neural network. It receives the input features and passes them to the hidden layers for processing.

Example:

A house price prediction model may use:

- Area
- Number of Bedrooms
- Age of House

as input features.

```
import tensorflow as tf

# Sample input data
inputs = tf.constant([[1200, 3, 10]])

print(inputs)
```

```
tf.Tensor([[1200    3   10]], shape=(1, 3), dtype=int32)
```

2. Hidden Layer:-

Hidden Layers perform computations on the input data and learn patterns by applying weights, biases, and activation functions.

A neural network may contain one or multiple hidden layers depending on the complexity of the problem.

```
import tensorflow as tf

hidden_layer = tf.keras.layers.Dense(8, activation="relu")
```

```
print(hidden_layer)
```

```
<Dense name=dense, built=False>
```

3. Output Layer:-

The Output Layer is the final layer of the neural network. It produces the prediction based on the processed information.

The number of neurons depends on the problem:

- 1 neuron for Regression
- 1 neuron with Sigmoid for Binary Classification
- Multiple neurons with Softmax for Multi-class Classification

```
import tensorflow as tf

output_layer = tf.keras.layers.Dense(1)

print(output_layer)
```

```
<Dense name=dense_1, built=False>
```

Building a Sequential Model:-

A **Sequential Model** is the simplest way to build a neural network in TensorFlow using Keras. It creates a model by stacking layers one after another in a linear sequence.

It is suitable for problems where the data flows from one layer directly to the next without branching.

1. Sequential():-

Sequential() is used to create a linear stack of layers in a neural network.

```
import tensorflow as tf

model = tf.keras.Sequential()

print(model)
```

```
<Sequential name=sequential, built=False>
```

2. Dense():-

Dense() creates a fully connected (dense) layer where every neuron is connected to every neuron in the previous layer.

Parameters:

- **units:** Number of neurons in the layer.
- **activation:** Activation function used by the layer.

```
import tensorflow as tf

layer = tf.keras.layers.Dense(units=64, activation="relu")

print(layer)
```

```
<Dense name=dense_2, built=False>
```

3. Creating a Simple Sequential Model:-

A Sequential model is created by adding layers in order. The first layer receives the input, hidden layers learn patterns, and the final layer produces the output.

Example :-

```
import tensorflow as tf

model = tf.keras.Sequential([
    tf.keras.layers.Dense(64, activation="relu", input_shape=(10,)),
    tf.keras.layers.Dense(32, activation="relu"),
    tf.keras.layers.Dense(1)
])

model.summary()
```

```
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/dense.py:106: U
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
Model: "sequential_1"
```

Layer (type)	Output Shape	Param #
dense_3 (Dense)	(None, 64)	704
dense_4 (Dense)	(None, 32)	2,080
dense_5 (Dense)	(None, 1)	33

```
Total params: 2,817 (11.00 KB)
Trainable params: 2,817 (11.00 KB)
Non-trainable params: 0 (0.00 B)
```

Model Compilation:-

Before training a neural network, the model must be **compiled**. Model compilation configures how the model will learn from the data.

The compile() method specifies:

- **Optimizer:** Updates the model's weights during training.
- **Loss Function:** Measures how well the model performs.
- **Metrics:** Evaluates the model's performance.

1. compile():-

The compile() method prepares the model for training by configuring the optimizer, loss function, and evaluation metrics.

```
import tensorflow as tf

model = tf.keras.Sequential([
    tf.keras.layers.Dense(16, activation="relu", input_shape=(4,)),
    tf.keras.layers.Dense(1, activation="sigmoid")
])

model.compile(
    optimizer="adam",
    loss="binary_crossentropy",
    metrics=["accuracy"]
)

print("Model compiled successfully!")
```

```
Model compiled successfully!
```

2. Optimizer:-

An **Optimizer** updates the model's weights to reduce the loss and improve prediction accuracy during training.

Some commonly used optimizers are:

- Adam
- SGD (Stochastic Gradient Descent)
- RMSprop

Example:

optimizer="adam"

```
import tensorflow as tf

optimizer = tf.keras.optimizers.Adam()
```

```
print(optimizer)
```

```
<keras.src.optimizers.adam.Adam object at 0x79b00964c740>
```

3. Loss Function:-

A **Loss Function** measures the difference between the actual output and the predicted output. The objective during training is to minimize the loss.

Some commonly used loss functions are:

- Mean Squared Error (Regression)
- Binary Crossentropy (Binary Classification)
- Categorical Crossentropy (Multi-class Classification)

Example:

loss="binary_crossentropy"

```
import tensorflow as tf

loss = tf.keras.losses.BinaryCrossentropy()

print(loss)
```

```
<LossFunctionWrapper(<function binary_crossentropy at 0x79b03bccc7c0>, kwargs
```

4. Metrics:-

Metrics are used to evaluate the performance of the model during and after training.

Common metrics include:

- Accuracy
- Precision
- Recall

Example:

metrics=["accuracy"]

```
import tensorflow as tf

metric = tf.keras.metrics.BinaryAccuracy()

print(metric)
```

```
<BinaryAccuracy name=binary_accuracy>
```

Model Training :-

After compiling the model, the next step is **training**. During training, the model learns patterns from the training data by adjusting its weights to minimize the loss function.

TensorFlow uses the `fit()` method to train a model.

Training Process

- The model receives input data.
- It makes predictions.
- The loss is calculated.
- The optimizer updates the weights.
- This process repeats for multiple epochs.

1. fit():-

The fit() method trains the model using the provided training data.

Parameters:

- **x:** Training input data.
- **y:** Target values.
- **epochs:** Number of complete passes through the training dataset.
- **batch_size:** Number of samples processed before updating the weights (optional).

```
import tensorflow as tf
import numpy as np

# Sample data
x = np.random.rand(100, 4)
y = np.random.randint(0, 2, size=(100, 1))

# Create model
model = tf.keras.Sequential([
    tf.keras.layers.Dense(8, activation="relu", input_shape=(4,)),
    tf.keras.layers.Dense(1, activation="sigmoid")
])

# Compile model
model.compile(
    optimizer="adam",
    loss="binary_crossentropy",
    metrics=["accuracy"]
)
```

```
# Train model
model.fit(x, y, epochs=5)
```

```
Epoch 1/5
4/4 ━━━━━━━━━━━━━━━━━━━━ 2s 11ms/step - accuracy: 0.5200 - loss: 0.8202
Epoch 2/5
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 11ms/step - accuracy: 0.5200 - loss: 0.8099
Epoch 3/5
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - accuracy: 0.5200 - loss: 0.8007
Epoch 4/5
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 11ms/step - accuracy: 0.5200 - loss: 0.7913
Epoch 5/5
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 11ms/step - accuracy: 0.5200 - loss: 0.7826
<keras.src.callbacks.history.History at 0x79b009688740>
```

2. Epochs:-

An **Epoch** means one complete pass of the entire training dataset through the neural network.

Increasing the number of epochs allows the model to learn better, but too many epochs may lead to **overfitting**.

Example:

epochs=10

```
import tensorflow as tf
import numpy as np

x = np.random.rand(50, 2)
y = np.random.randint(0, 2, size=(50, 1))

model = tf.keras.Sequential([
    tf.keras.layers.Dense(4, activation="relu", input_shape=(2,)),
    tf.keras.layers.Dense(1, activation="sigmoid")
])

model.compile(
    optimizer="adam",
    loss="binary_crossentropy"
)

model.fit(x, y, epochs=3)
```

```
Epoch 1/3
2/2 ━━━━━━━━━━━━━━━━━━━━ 2s 74ms/step - loss: 0.7007
Epoch 2/3
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 38ms/step - loss: 0.6999
Epoch 3/3
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 51ms/step - loss: 0.6994
<keras.src.callbacks.history.History at 0x79b0053579e0>
```

Model Evaluation:-

After training a model, it is important to evaluate how well it performs on unseen data. **Model Evaluation** helps determine whether the model has learned meaningful patterns or is simply memorizing the training data.

TensorFlow provides the **evaluate()** method to measure the model's performance using test data.

1. evaluate():-

The `evaluate()` method calculates the loss and performance metrics of a trained model using test data.

Returns:

- Loss Value
- Evaluation Metrics (such as Accuracy)

Example Output:

Loss: 0.8160926699638

Accuracy: 0.400000005960

2. Loss:-

The **Loss** value indicates how far the model's predictions are from the actual values.

- Lower loss indicates better performance.
- During training, the goal is to minimize the loss.

Example Output:

Loss: 0.45

3. Accuracy:-

Accuracy measures the percentage of correct predictions made by the model.

It is one of the most commonly used evaluation metrics for classification problems.

Formula:

Accuracy = Correct Predictions / Total Predictions

Example Output:

Accuracy: 0.92

Predictions :-

Once a model has been trained and evaluated, it can be used to make predictions on new, unseen data. Predictions are the final output of a machine learning model and are used in real-world applications such as image classification, spam detection, and house price prediction.

TensorFlow uses the `predict()` method to generate predictions.

1. predict() :-

The `predict()` method uses a trained model to predict outputs for new input data.

Syntax:

```
model.predict(new_data)
```

Returns:

- Predicted values
- Probabilities (for classification models)

```
import tensorflow as tf
import numpy as np

model = tf.keras.Sequential([
    tf.keras.layers.Dense(1, input_shape=(2,))
])

model.compile(optimizer="adam", loss="mse")

data = np.array([[5, 6]])

prediction = model.predict(data)

print(prediction)
```

```
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 98ms/step
[[0.42072248]]
```

2. Predicting a Single Sample:-

You can also use the `predict()` method to predict the output for a single input sample.

The input must still be provided as a two-dimensional array.

Example:

```
sample = np.array([[0.5, 0.2, 0.8, 0.1]])
model.predict(sample)
```

```
import tensorflow as tf
import numpy as np

model = tf.keras.Sequential([
    tf.keras.layers.Dense(1, input_shape=(2,))
])

model.compile(optimizer="adam", loss="mse")

data = np.array([[5, 6]])

prediction = model.predict(data)

print(prediction)
```

```
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 241ms/step
[[2.3667998]]
```

Saving & Loading Models :-

After training a model, it is often saved so it can be used later without retraining. TensorFlow provides simple methods to save and load trained models.

Saving models helps in deploying machine learning applications and sharing trained models.

1. save():-

The save() method saves the entire trained model to a file.

Example:

```
model.save("my_model.keras")
```

```
import tensorflow as tf

model = tf.keras.Sequential([
    tf.keras.layers.Dense(1, input_shape=(2,))
])

model.save("my_model.keras")
```

2. load_model():-

The load_model() function loads a previously saved model.

Example:

```
tf.keras.models.load_model("my_model.keras")
```

```
import tensorflow as tf

model = tf.keras.models.load_model("my_model.keras")

print(model)
```

```
<Sequential name=sequential_7, built=True>
```

Conclusion :-

TensorFlow is one of the most powerful and widely used libraries for Machine Learning and Deep Learning. It provides tools for creating tensors, performing mathematical operations, building neural networks, training models, evaluating performance, making predictions, and saving trained models.